



Technical Project Setup

The following tools are recommended, but not strictly necessary for work.

Still, you may benefit from reading this page even if you have a workflow already.

If you are new to programming, follow the instructions as they **appear** in the guide.

1. Introduction

We assume a basic familiarity with opening the terminal app and being able to run commands.

Being able to call a script, and a function like `ls` and `cd` should be enough, but `find`, `nano`, `grep`, `cat`, `>`, `>>`, `|` are also extremely useful and worth learning. We recommend **learning** from some videos: [Unix basics](#), [Complete Guide - Strongly Recommended](#).

The terminal is more reliable in Unix (Macos and Linux) operating systems: some tools may have to be installed, and, in general, things will look “hacky” on Windows.

2. Package Manager

Since coding relies on multiple programs, you should use a single package manager to manage them, and avoid as much as possible manual installation. A package manager is a program acting as your app store, which can be used to update all apps at once, remove one completely, or download a new one. It also downloads dependencies in order to guarantee that the app runs on the first try. You may also want to install specific versions of a program, such as old ones or versions still in development. Typically, you should check the exact name of a package to install the correct one.

You should have a single manager and use only that to install, update and remove apps. This guarantees that nothing breaks over time, as apps are not forgotten.

Install the package manager for your operating system:

- **MacOS** :: [Homebrew](#)
 - `brew install <app>` :: installs the app by name. Some apps have a `--cask` option, which makes you download the GUI.
 - `brew list` :: lists all downloaded apps.
 - `brew uninstall <app>` :: uninstalls an app, together with its dependencies.
 - `brew upgrade` :: actually upgrades installed packages.
 - `brew update` :: updates Homebrew itself and the formulae metadata.
- **Windows** :: [winget](#)
 - `winget install <app>` :: installs the app by name (you can also use `--id <Package.Id>` for precision).
 - `winget list` :: lists all installed apps that winget can see.
 - `winget uninstall <app>` :: uninstalls an app.
 - `winget upgrade --all` :: upgrades all upgradable apps.
- **Linux** :: the default package manager depends on the distribution, and updates everything, from the operating system to the drivers.

You can check if a program is installed by opening a new terminal page and writing its name and pressing enter. If it says: “command not found”, it is not installed. Sometimes names differ:

VSCoDe’s terminal *alias* is `code`; for example, `code .` will open the current folder using VSCoDe.

Package managers are at least as safe as downloading the files directly: they check file integrity using [checksums](#), but they still require you to trust the entity that creates the code.

The guide assumes that your computer has no program installed already. If some of the tools are already installed on your device, you may keep them. They could stop working eventually. At that point, you should install them correctly after uninstalling them.

3. git

[Git](#) is the tool for managing different versions of text files that you are working on, from documentation files (`latex`, `typst`, `html`) to code in any programming language, including notebook (`jupyter`) files. You should also store other files through git, like small images, but you should not use it to save big files, such as training data, as git is made for managing code. Git-inspired like `restic` (for backups) and `git LFS` exist, but they are not relevant to this guide.

With git, you are able save checkpoints called *commits* into a tree structure. You may also work on different versions of the code at the same time, separating the development process into *branches*, to be merged when ready. This is particularly useful in a team. Some advantages to this workflow are the ability to implement big changes in small and revertible steps, as well as being able to identify the few lines that broken a feature by looking back in time and finding the specific commit that introduced a bug. There are not many good reasons not to use Git.

Git (software) should not be confused with Github (website), which is commonly used to store git *repositories* online. Github was bought by Microsoft for the data it contains, which is used to train *Github Copilot*, a lucrative service that gets sold back to Github's users using open source code developed by a community to be a common good.

You should **install** git through your package manager. You can check that it is installed by typing `git`. Although code editors make it easier to interact with its main features, you can access all of them from the terminal.

In general, git works by remembering on what branch you are, and which was the last commit.

- `git clone <link>` :: can be used to download to a new folder an existing project, using its link.
- `git status` :: shows the last commit and the current status of the repository.
- `git init .` :: creates a git repository in the current directory
- `git add .` :: adds all files to the *staged* area, meaning “about to be committed”.
- `git commit -m "<message>"` :: saves the staged changes as a commit with the “”.
 - [Some commit advice](#).
- `git reset --soft HEAD^` :: undoes the last commit command, while keeping staged files and unstaged files. To correct a local commit, do a soft reset. If you have pushed, the simplest option is pushing a new commit. Alternatively, you can do `git push -f` if you know that overwriting later commits is safe.
- `git reset --hard HEAD^` :: CAREFUL, resets the current state of the repository to the last commit. **You will discard the last changes since the commit.**

Git is often used in conjunction with an *origin*, a centralized storage area for the repository. This is the role of Github, but private repositories may also be set up.

- `git push` :: pushes to the upstream branch the commits that were added locally, unless someone else did it first. In that case, you should soft-reset the commit, pull, fix the conflicts, then commit and push.
- `git fetch` :: downloads the latest changes from the origin.
- `git pull` :: if possible, jumps to the latest state of the current repository. If your commit is the penultimate and the branch in origin shows a later commit, running the command will update your current state to the latest.

During collaborative work, you should always push everything before closing your laptop.

- `git branch -a` :: list all branches
 - `git switch <branch>` :: switches to an existing branch.
 - `git switch -c <branch>` :: creates a new branch and switches to it.
- `git restore <path>` :: restores file(s) in the working tree to the last committed state.
 - `git restore --staged <path>` :: unstages file(s), keeping changes in the working tree.
 - `git switch --detach <commit_hash>` :: (checkout that commit in a detached HEAD state).
- `git merge <branch>` :: merges “branch” to the current one.

You should maintain a `.gitignore` file in a git repository in order to tell git which files to ignore. Some typical lines you can use as reference are the following:

- `*.pdf` :: all pdf files.
- `foldername/` :: all files inside folder “foldername”.
- `**/foldername/` ignore all folders “foldername”, no matter in which subfolder they are contained.

Git is extremely safe when you use it correctly, but think before running commands, as you may lose the last changes or even full commits otherwise. During a crisis, refer to `git reflog`.

3.1. Setup

You can pair your `git` with a [Github](#) account using the `gh` utility, which is developed by Github. Naturally, you must first **sign up** to Github [here](#). Next, **install** `gh` using your package manager. Afterwards, type “gh auth login”; press enter four times, which selects the default options, then follow the instructions that appear.

As a last step, **set** your name and mail through the terminal app, so git can use them. **Verify** they are set correctly with the following commands: write them in the terminal and press **enter** to execute the instruction.

```
git config --global user.name "<Your Name>"
git config --global user.email "<you@example.com>"
```

You can check that `user.name` and `user.email` are set with the following commands, which should repeat the values as they were saved.

```
git config --get user.name
git config --get user.email
```

You should also set: `git config --global pull.ff only`. Here’s [why](#).

We recommend installing the VSCode extension [git-graph](#) and to set the command `git adog` and use that. Just do once this command, and `git adog` will work aways:

```
git config --global alias.adog "log --all --decorate --oneline --graph".
```

4. Visual Studio Code

[VSCode](#) is the standard code editor. It supports virtually all programming languages through the installation of *extensions*, and comes ready out of the box. You can **install** it with your package manager, and it interfaces automatically with git. VSCode is also owned and developed by Microsoft, as it allows the company to collect metrics on how developers write code.

There are also smaller and more trusted code editors such as Neovim, with just as much support, but as a first code editor, VSCode is the best.

There are some **customizations** you may want to do:

- **theme** :: light, dark and many more.
- **Autosave** :: enable it, so files are saved after every changes, and in addition to git, you never risk losing work.
- **extensions** :: install **Prettier**, **Pylance**, **Code Spell Checker**, and any other extension for your commonly used languages. You should use a code formatter to frequently standardize the structure of the code, as it makes git differences more meaningful.
- VSCode works through commands: you can search for one by name using **(ctrl|cmd)-shift-p**.

Since you can configure VSCode to work with any formatting language, you could / should also develop your CV with it, relying on git. This can be a learning exercise with a payoff.

5. Python

[Python](#) is a language with features that make it fast to develop, slow to run and somewhat hard to verify. All in all, this has made it the preferred language for ML.

To **install** Python,

- If this is the first time, install **uv** and let it handle downloading python on its own.
- If you do not intend to use **uv**, or are using **conda** now, use your package manager. Note that Python comes in versions, which are not always compatible: 3.13 or 3.14 are fine, unless you need to use an older one. Large numerical libraries generally lag behind, using previous versions.

You can run python from the shell using **python**, which opens the interactive python [REPL](#). You can check the python version using **python --version**.

Some important libraries:

- **Linear Algebra**: **Numpy** is the standard library.
- **DataFrames**: **Pandas** is more widely supported, but **Polars** also exists, which has a lower energy footprint and is a lot faster.
- **Graph Theory**: **networkx** is the standard. Some integrations exist such as **osmnx**.
- **Machine Learning**: **Pytorch** for NN (with ecosystem: **torchvision**, **torchaudio**, **torchtext**), **scikit-learn** for simple things like clustering or decision trees, **scipy** for optimization, signal processing, distributions.
- **Better use of GPU**: apply gpu support through the ML library you are using.
- **Faster Python** for small numerical functions: **Numba**. Things like **Cython** and **PyPy** also exist, but at that point you may as well use a lower-level language like **C**, **Go** or **Rust**.

You should also use the **tqdm** [package](#) to track completion level of large loops like training.

If you want to go deeper into programming, you may want to try a language that does not hide how the computer works. For a language which is fast and easy to debug, see [Rust](#).

6. UV venv

Just like your computer needs a package manager for the apps, most programming languages need a package manager for their libraries. This comes from the fact that languages have versions, and each version is compatible with only some versions of the libraries. The solution is to create a *virtual environment*, in which a version of the language is fixed, and all the necessary packages are installed, keeping track of their versions and compatibility for future reference. This ensures that code is reproducible. Packages are thus saved in a folder inside the project, which is generally made to start with a dot (**.venv**, **.venv-gpu**), so the folder is hidden by the file explorer, in the same way as the **.git** folder. You should never edit these folders directly.

Said briefly: *pip* is the default Python package manager; *conda* is commonly recommended over *pip* since it avoids copying the package files every time the same file is needed, and it tries to be the all-in-one solution, but it is very slow and not too useful.

We recommend **installing** [Atrai's UV](#). It is less known, but it is 10-100x faster (literally), simple to use and easier to work with. As before, use your package manager.

- `uv init .` :: creates a new project in the current folder.
 - `uv init <folder>` :: creates a subfolder called “<folder>” with a new project in it.
- `uv add <package>` :: adds a new package with its dependencies.
- `uv remove <package>` :: removes a new package with its direct dependencies.
- `uv sync` :: looks at the *pyproject.toml* file and installs the packages that should be downloaded but are not.
- `uv run script.py` :: runs `script.py` with the correct python version and its packages.

For example, a member of the team may realize that `seaborn` is needed: they run `uv add seaborn`, then commit the change. Once the other people do `git pull`, then can just `uv sync` to immediately include the new package. Easy fix.

6.1. Moving from Conda

Minimal change: run `conda config --set auto_activate_base false` to stop the automatic enabling of the base environment. This does not delete Anaconda.

If you want to migrate a conda environment to a uv managed one:

1. `conda list -e > requirements.txt` then do `conda deactivate`.
2. `uv init .` then import the requirements with `uv pip install -r requirements.txt`.

Once you get comfortable with uv, you are welcome to uninstall conda.

6.2. Moving from pip/venv

Minimal change: you don't need to uninstall anything. Just stop creating new virtualenvs with `python -m venv` and start using `uv` for new projects.

If you want to migrate an existing pip/venv project to a uv-managed one:

1. Activate your current virtualenv, run `pip freeze > requirements.txt` and export its packages `deactivate`.
2. `uv init .` then import the requirements with `uv pip install -r requirements.txt`.

Once you're comfortable with uv, you can stop using `venv` for new projects (and optionally remove old virtualenvs when you no longer need them).

6.3. Jupyter Notebooks

[Notebook](#) files are a powerful tool for development, as they allow for the execution of blocks of code in an interactive environment which alternates markdown text with python code.

VSCoDe offers top tier support for Jupyter notebooks through the `Jupyter` extension. To use an environment in a notebook, you must **add** the `ipykernel` package in the venv. After this step, you can select the environment from the notebook, and use it in the notebook.

It is good practice for each block in a notebook to carry out an operation based on the output of the previous block, and save the result of this operation into a new variable. This guarantees that each block can be run multiple times, without overwriting the results of the previous blocks.

Moreover, whenever you generate a plot, the title of the plot should always contain the specific setting and parameters, so you can recreate the result easily.

Development of new python code can be carried out quickly in a notebook:

1. Write code that does some operation on one input.

2. Turn the code into a function that takes a generic input.
3. Move the function to a python file and import it with `from filename import *`. This step results in a short notebook, with python files that can be imported and used as a library.

Git is not the best at handling notebooks, so in order to guarantee the quality and reproducibility of the code, always restart the notebook kernel and run the whole notebook in one step (do **Run All**) before committing a new version. This guarantees that blocks are consistent and can run without errors. In addition, since images or text outputs can grow large, a good idea may also be to remove some outputs from a notebook, to prevent the repository from growing large.

On a sidenote, the Jupyter notebook format also supports some other languages such as **Julia** and **SageMaths**, and similar versions exist in languages like **R** with its **.Rmd** format.

7. Notetaking

You should write notes on the project itself, in the repository, as you are doing it: it keeps everyone up to date with the results and future direction, and removes the struggle at the very end to remember all the steps of development. Most import, at the end, the report is just a minor reformat away from the notes. The report should also be version controlled, ideally with git, so it can be stored with the other files in the repository. Text files, written in a [markup language](#), are the best option for this purpose.

You may want to rely on two files: one for notes and resources such as links, and the other for the writeup of the results. A typical workflow should also involve an **img/** folder for storing the images that will go in the report.

Latex or Markdown are commonly used to write notes. Since they both have shortcomings (Latex is slow and somewhat difficult to set up, while markdown does not provide much formatting options), we suggest Typst as the easy all-in-one tool.

- [Markdown](#) is the most minimal option for storing the notes in the repo. It also has limited *html* support, as you can see from some fancy README files on Github. It is used by default in README files. You can easily setup markdown to work in VSCode by **installing** the **Markdown All in One** extension, together with an instant preview option, provided by the **Markdown Preview Enhanced** extension. You cannot use Markdown to produce a good a final report: you will have to paste the text into another and convert the format.
- [Latex](#) is the standard formatting language for technical research papers, but it is somewhat non-obvious to configure: to install it, it may or may not be a good idea to rely on the package manager, but if you can do it, it is probably the best option. On the other hand, students are not as knowledgeable, so some freemium options like: [Overleaf](#) is a commonly used, which has limited support for version control. You may also find [Detexify from drawing](#) useful.

If you are writing notes in VSCode in any markup language, you should **install** the **LTEX+ Extension** for VS Code to do spell checking while you write.

7.1. Typst

We recommend installing [Typst](#) as the best of two worlds: it is simpler to use and faster to interact with than Latex, and works out of the box with an easy installation like Markdown. It can be used to write notes and complete documents (such as this one), with the only drawback being less support from within language models; we provide the cheat sheet [comparison with Latex](#) and links to the [Typst Documentation](#) to remedy this.

Typst documents can easily replace Latex at a smaller time cost. You can set the Typst font to the Latex default using: `#set text(font: "New Computer Modern", size: 11pt)`.

For writing formulas, Typst is [similar to Latex, but simpler](#):

```
$ cal(A) := { x in RR | x "is natural" } $
$ cos(x) = lim_(n -> oo) sum_(k=0)^n (-1)^k x^(2k)/(2k)!$
```

$$\mathcal{A} := \{x \in \mathbb{R} \mid x \text{ is natural}\}$$

$$\cos(x) = \lim_{n \rightarrow \infty} \sum_{k=0}^n (-1)^k \frac{x^{2k}}{(2k)!}$$

```
$$\mathcal{A} := \{ x \in \mathbb{R} \mid x \text{ is natural} \} $$
$$\cos x = \lim_{n \rightarrow \infty} \sum_{k=0}^n (-1)^k \frac{x^{2k}}{(2k)!} $$
```

You can **install** Typst with your package manager: `brew install typst` or `winget install typst`. Next, **install** the `Tinymist Typst` extension in VSCode, which supports instant preview (side by side scrolling + click the preview to jump to the exact word in the source, and vice versa), as well as drawing symbols to obtain their word. When writing a file, you can enable the former with `Preview Opened File`, while the latter is a left-side menu.

8. Project Structure

You have to **organize** the files of the project, so the folder is easy to work with.

For large or frequently changed files such as results of experiments:

- For the dataset(s), use a `data/` folder and ignore it with `.gitignore`. Use an `img/` folder to store images and pictures that you want to keep in the final report. Finally, the results from code running go in an `out/` folder, also ignored by git. This separation ensures that git history is light, important results are stored, and code never risks overwriting the dataset. Consider also ignoring `__pycache__` and other files.

For code and for journaling:

- As you write more code, move “utility functions” to [self-sufficient files](#), where each file contains all the methods necessary for a specific operation of the full process. If data cleaning happens in multiple steps, jupyter notebooks could be named “1. ...”, “2. ...” to sort the files.
- Somewhat rarely, it can be a good idea to store specific documentation (eg. pdfs) in the repository for quick access and sharing to the members. In other cases, the opposite: messy users could benefit from a `.gitignore` that ignores all files, except the `gitignore` itself and python files. This avoids cluttering the `git status` with useless untracked files.
- Create the `Project_Report.[tex|typ]` file, to record the discoveries as you make them, referencing notable discoveries from the `img/` folder.

Remember to **commit** the empty project structure into an “Initial Commit”. This can be useful when starting out a new project.

9. LLMs

Large Language models are impressive at doing short tasks and reading documentation. If you use them repeatedly for longer tasks, you need to give very detailed instructions, or the code will become longer at each iteration and decrease in quality. Not only that, as the code gets longer you begin to waste more time, to the point where the advantages of using an LLM instead of reading the documentation get lost. Never forget that language models are not good at telling you how much complexity is too much, so if you do not know what you are doing, the code will quickly get over-engineered and incredibly hard to maintain. On a final note, if all that you do is prompts, are you trying to make yourself replaceable? **Always** use your skills to the most, so you always learn.